EECE 7205: Fundamentals of Computer Engineering
Fall 2026
Assignment #02

Paulina Skurzak
NUID: 003973748
skurzak.p@northeastern.edu
Instructor: Dr. Fatema Nafa

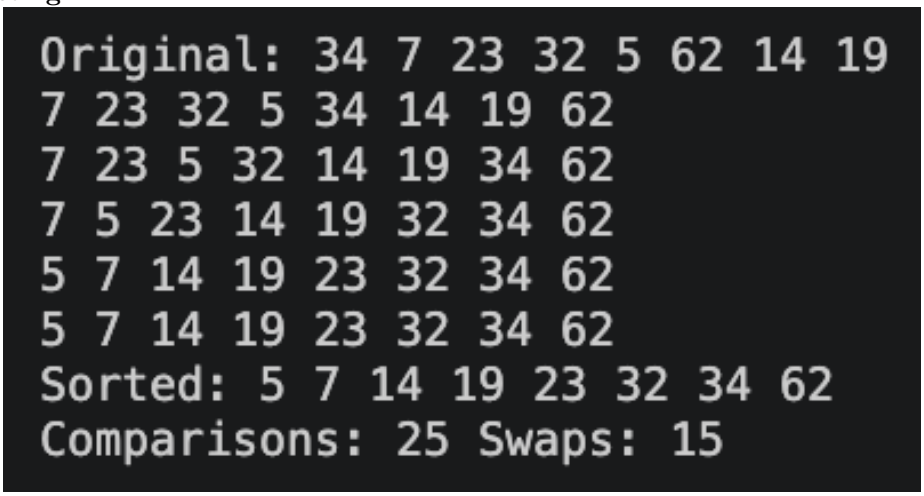
Problem 1 [Bubble Sort]

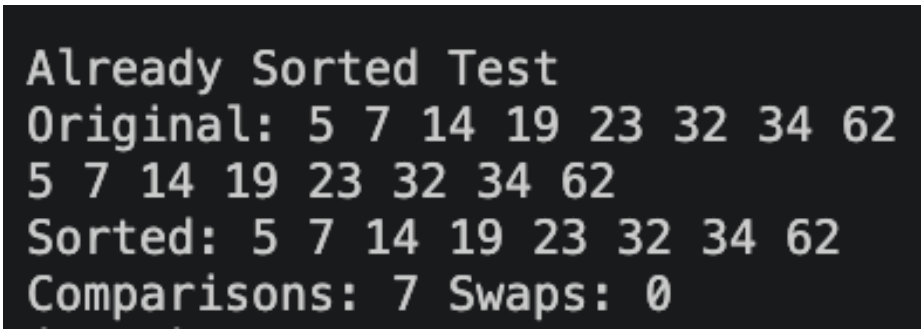
Pseudocode:

```
BUBBLE-SORT(A, n, comparisons, swaps)
n <- SizeOf(A)
for Pass <- 0 to n - 2 do
    swapped <- FALSE
    for i <- 0 to n - Pass - 2 do
        comparisons <- comparisons + 1
        if A[i] > A[i+1] then
            SWAP(A[i], A[i+1])
            swaps <- swaps + 1
            swapped <- TRUE
        end if
    end for
    if swapped = FALSE then
        break
    end if
end for
return A
```

Solution Approach: The bool swapped flag starts as false at the beginning of every pass. It essentially assumes nothing will need to move, until proven otherwise. It changes to true after a swap occurs during a pass. It is checked after a pass finishes and if it is still false, that means every single adjacent pair was already in correct order, meaning the whole array is already sorted. This is what allows the algorithm to stop early instead of running the full number of passes. The nested-loop structure in this algorithm allows for reseting the bool swapped flag once per pass rather than per comparison.

Testing:





Question: Run your program again on an already sorted copy of A. How many passes and comparisons does it make now? Explain why in your report.

The sorted array only needs 1 pass and 7 comparisons. Since there are 8 total elements, one full pass through the array makes 7 adjacent comparisons. Only 1 pass is required because the array is already in order, so no swaps occur during that pass. This means the bool swapped flag remains false, triggering the early exit and preventing any additional passes.

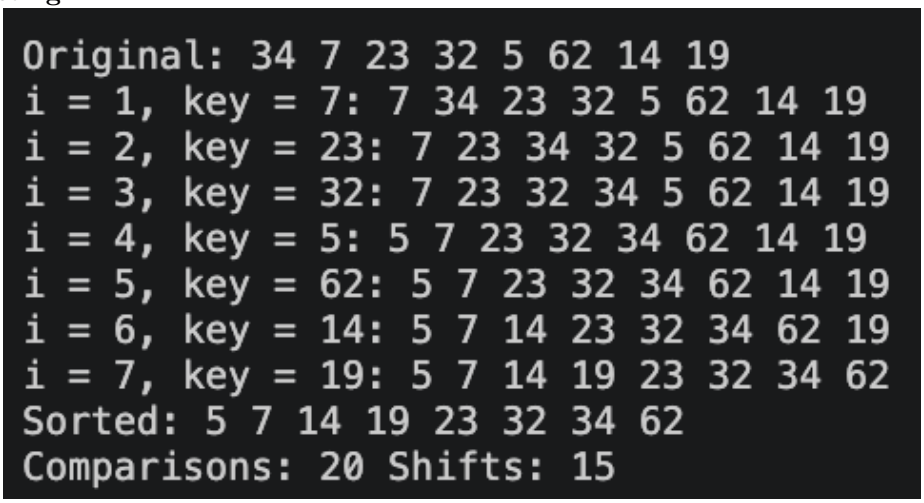
Problem 2 [Insertion Sort]

Pseudocode:

```
INSERTION-SORT(A, n, comparisons, shifts)
n <- SizeOf(A)
for i <- 1 to n - 1 do
    key <- A[i]
    j <- i - 1
    while j >= 0 and A[j] > key do
        comparisons <- comparisons + 1
        A[j+1] <- A[j]
        shifts <- shifts + 1
        j <- j - 1
    end while
    if (j >= 0) comparisons <- comparisons + 1
    A[j+1] <- key
end for
return A
```

Solution Approach: The key stays the same value the whole time, while the loop shifts bigger elements to the right to make room for it. Once the loop stops, the key gets placed into its correct spot. I also added one extra comparison count right after the loop ends. The loop can stop for two different reasons. If j goes below 0, nothing was left to check, so no real comparison happened. But if j is still 0 or higher, a real comparison did happen, it just never got counted inside the loop, since the loop only counts comparisons when it actually shifts something. So I added a check afterward to count that comparison too.

Testing:



```
 Fewest Shifts Test
Original: 5 7 14 19 23 32 34 62
i = 1, key = 7: 5 7 14 19 23 32 34 62
i = 2, key = 14: 5 7 14 19 23 32 34 62
i = 3, key = 19: 5 7 14 19 23 32 34 62
i = 4, key = 23: 5 7 14 19 23 32 34 62
i = 5, key = 32: 5 7 14 19 23 32 34 62
i = 6, key = 34: 5 7 14 19 23 32 34 62
i = 7, key = 62: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 7Shifts: 0

 Most Shifts Test
Original: 62 34 32 23 19 14 7 5
i = 1, key = 34: 34 62 32 23 19 14 7 5
i = 2, key = 32: 32 34 62 23 19 14 7 5
i = 3, key = 23: 23 32 34 62 19 14 7 5
i = 4, key = 19: 19 23 32 34 62 14 7 5
i = 5, key = 14: 14 19 23 32 34 62 7 5
i = 6, key = 7: 7 14 19 23 32 34 62 5
i = 7, key = 5: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 28 Shifts: 28
```

Question: Which ordering of the same eight numbers makes Insertion Sort do the fewest shifts, and which makes it do the most? Run your program on both orderings, report the counts, and explain the result.

The already sorted array needs the fewest shifts, 0, because every element is already in its correct position, so no changes need to be made. The reverse sorted array needs the most shifts, 28, because each new key has to pass every previously placed element, since they are all bigger than it.

Problem 3 [Selection Sort]

Pseudocode:

```
SELECTION-SORT(A, n, comparisons, swaps)
n <- SizeOf(A)
for i <- 0 to n - 2 do
    minIdx <- i
    for j <- i + 1 to n - 1 do
        comparisons <- comparisons + 1
        if A[j] < A[minIdx] then
            minIdx <- j
        end if
    end for
    if minIdx != i then
        SWAP(A[i], A[minIdx])
        swaps <- swaps + 1
    end if
end for
return A
```

Solution Approach: Selection Sort works by finding the minimum value in the unsorted portion of the array, then swapping it into place at the front. Once a minimum is found and placed, its correct position is guaranteed and does not need to be looked at further, so the algorithm shrinks the unsorted portion by one and repeats.

Testing:

```
Original: 34 7 23 32 5 62 14 19
min = 5 at index 4: 5 7 23 32 34 62 14 19
min = 7 at index 1: 5 7 23 32 34 62 14 19
min = 14 at index 6: 5 7 14 32 34 62 23 19
min = 19 at index 7: 5 7 14 19 34 62 23 32
min = 23 at index 6: 5 7 14 19 23 62 34 32
min = 32 at index 7: 5 7 14 19 23 32 34 62
min = 34 at index 6: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 28 Swaps: 5
```

Question: Compare the number of swaps with your result for Bubble Sort in Problem 1. Which algorithm makes fewer swaps on A, and why?

Selection Sort makes fewer swaps compared to Bubble Sort. This is because Selection Sort only performs at most one swap per pass, once the minimum value for that pass has been found. Bubble Sort, on the other hand, can perform

multiple swaps within a single pass, since it swaps every adjacent pair it finds out of order as it moves through the array.

Problem 4 [Quick Sort]

Pseudocode:

```
QUICKSORT(A, low, high, comparisons, swaps)
procedure QUICKSORT(A, low, high, comparisons, swaps)
    if low < high then
        p <- PARTITION(A, low, high, comparisons, swaps)
        QUICKSORT(A, low, p - 1, comparisons, swaps)
        QUICKSORT(A, p + 1, high, comparisons, swaps)
    end if
end procedure

procedure PARTITION(A, low, high, comparisons, swaps)
    pivot <- A[high]
    i <- low - 1
    for j <- low to high - 1 do
        comparisons <- comparisons + 1
        if A[j] <= pivot then
            i <- i + 1
            SWAP(A[i], A[j])
            swaps <- swaps + 1
        end if
    end for
    SWAP(A[i+1], A[high])
    swaps <- swaps + 1
    return i + 1
end procedure
```

Solution Approach: Quick Sort picks the last element as the pivot and partitions the array so smaller elements end up on its left and larger elements on its right. The pivot then sits in its final sorted position, and the algorithm recursively repeats this on the left and right sides.

Testing:

```
Original: 34 7 23 32 5 62 14 19
pivot: 19, subarray: 34 7 23 32 5 62 14 19
i = 0, j = 1, pivot = 19
7 34 23 32 5 62 14 19
i = 1, j = 4, pivot = 19
7 5 23 32 34 62 14 19
i = 2, j = 6, pivot = 19
7 5 14 32 34 62 23 19
Final position of pivot: 3
  pivot: 14, subarray: 7 5 14
    pivot: 5, subarray: 7 5
  pivot: 32, subarray: 34 62 23 32
    pivot: 62, subarray: 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 14 Swaps: 12
```

```
 Already Sorted Array Test
Original: 5 7 14 19 23 32 34 62
pivot: 62, subarray: 5 7 14 19 23 32 34 62
i = 0, j = 0, pivot = 62
5 7 14 19 23 32 34 62
i = 1, j = 1, pivot = 62
5 7 14 19 23 32 34 62
i = 2, j = 2, pivot = 62
5 7 14 19 23 32 34 62
i = 3, j = 3, pivot = 62
5 7 14 19 23 32 34 62
i = 4, j = 4, pivot = 62
5 7 14 19 23 32 34 62
i = 5, j = 5, pivot = 62
5 7 14 19 23 32 34 62
i = 6, j = 6, pivot = 62
5 7 14 19 23 32 34 62
Final position of pivot: 7
  pivot: 34, subarray: 5 7 14 19 23 32 34
    pivot: 32, subarray: 5 7 14 19 23 32
      pivot: 23, subarray: 5 7 14 19 23
        pivot: 19, subarray: 5 7 14 19
          pivot: 14, subarray: 5 7 14
            pivot: 7, subarray: 5 7
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 28 Swaps: 35
```

Question: Run your program on an already sorted copy of A and count the comparisons again. Explain why a sorted array is the worst case when the last element is the pivot, and suggest one way to avoid it.

Comparisons increase from 14 to 28 on the sorted array. A sorted array is the worst case because picking the last element as the pivot means every other element ends up smaller than it, so one side of the split gets all 7 remaining elements and the other side gets 0, as shown in the indented output above. This means the array only shrinks by one element at each level instead of splitting in half, which leads to $\Theta(n^2)$ time instead of $\Theta(n \log n)$. One way to avoid this is to pick a random element as the pivot instead of always using the last one.

Problem 5 [Merge Sort]

Pseudocode:

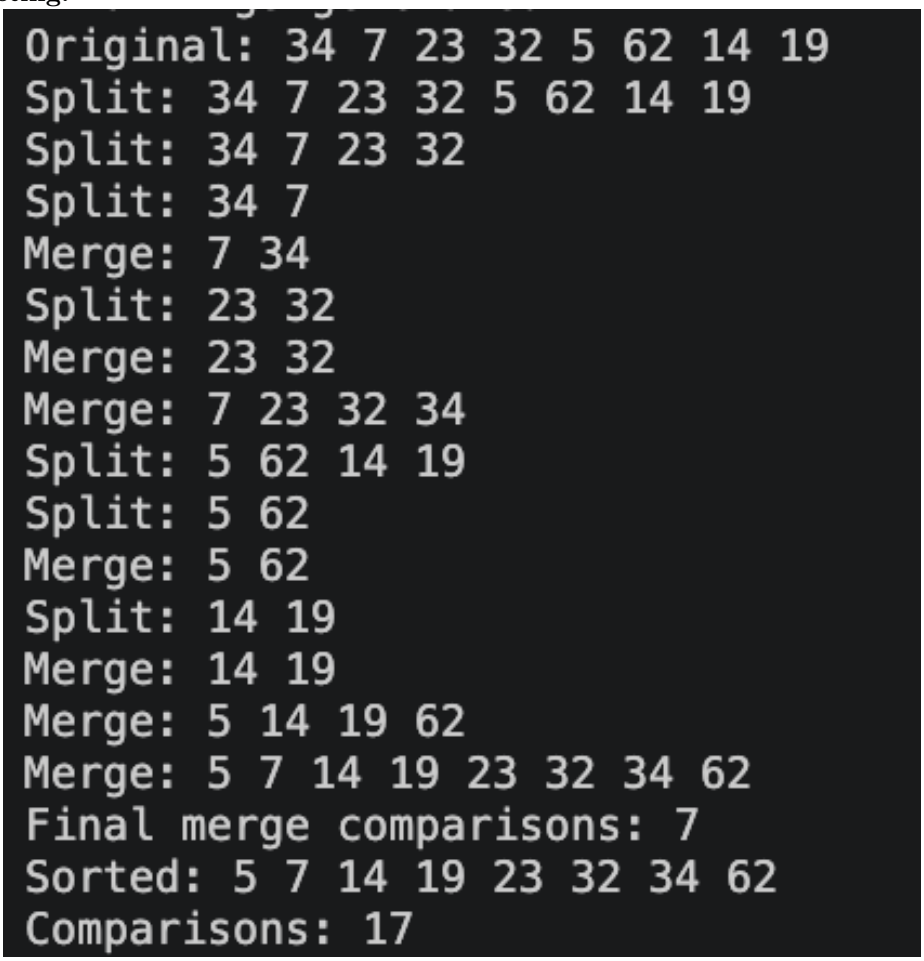
```
MERGESORT(A, low, high, comparisons, n,
    finalMergeComparisons)
procedure MERGESORT(A, low, high, comparisons, n,
    finalMergeComparisons)
    if low < high then
        mid <- floor((low + high) / 2)
        MERGESORT(A, low, mid, comparisons, n,
            finalMergeComparisons)
        MERGESORT(A, mid + 1, high, comparisons, n,
            finalMergeComparisons)
        MERGE(A, low, mid, high, comparisons, n,
            finalMergeComparisons)
    end if
end procedure

procedure MERGE(A, low, mid, high, comparisons, n,
    finalMergeComparisons)
    L <- A[low...mid]
    R <- A[mid+1...high]
    i <- 0, j <- 0, k <- low
    while i < SizeOf(L) and j < SizeOf(R) do
        comparisons <- comparisons + 1
        if low = 0 and high = n - 1 then
            finalMergeComparisons <- finalMergeComparisons +
                1
        end if
        if L[i] <= R[j] then
            A[k] <- L[i]
            i <- i + 1
        else
            A[k] <- R[j]
            j <- j + 1
        end if
        k <- k + 1
    end while
    while i < SizeOf(L) do
        A[k] <- L[i]
        i <- i + 1, k <- k + 1
    end while
    while j < SizeOf(R) do
        A[k] <- R[j]
        j <- j + 1, k <- k + 1
    end while
end procedure
```

Solution Approach: Merge Sort keeps splitting the array in half until each

piece only has one element left, then merges everything back together in sorted order. Since the array always splits exactly in half no matter what order the input is in, Merge Sort performs the same way regardless of how the numbers are arranged. I added a separate counter to track comparisons only for the final merge, the one that combines everything back into the whole array. I found this specific merge by checking if the current call's low and high match the original array's full range.

Testing:



```
 Sorted Array Test
Original: 5 7 14 19 23 32 34 62
Split: 5 7 14 19 23 32 34 62
Split: 5 7 14 19
Split: 5 7
Merge: 5 7
Split: 14 19
Merge: 14 19
Merge: 5 7 14 19
Split: 23 32 34 62
Split: 23 32
Merge: 23 32
Split: 34 62
Merge: 34 62
Merge: 23 32 34 62
Merge: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 12

 Reversed Array Test
Original: 62 34 32 23 19 14 7 5
Split: 62 34 32 23 19 14 7 5
Split: 62 34 32 23
Split: 62 34
Merge: 34 62
Split: 32 23
Merge: 23 32
Merge: 23 32 34 62
Split: 19 14 7 5
Split: 19 14
Merge: 14 19
Split: 7 5
Merge: 5 7
Merge: 5 7 14 19
Merge: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
```

Question: Run your program on a sorted copy and on a reversed copy of A. Explain why the number of comparisons changes much less than for Bubble Sort or Quick Sort, and state how much extra memory Merge Sort needs.

The number of comparisons only changes a little, from 17 on the original array to 12 on both the sorted and reversed copies. This is because Merge Sort always splits the array exactly in half at every level, no matter what order the numbers start in. This is different from Bubble Sort or Quick Sort, since their behavior depends a lot on how the elements are arranged. Merge Sort needs O(n) extra memory, since it makes temporary left and right arrays every time it merges.

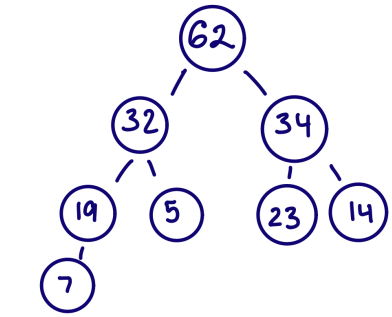
Problem 6 [Heap Sort]

Pseudocode:

```
HEAPSORT(A, n, swaps)
procedure HEAPSORT(A, n, swaps)
    n <- SizeOf(A)
    for i <- floor(n/2) - 1 downto 0 do
        SIFTDOWN(A, i, n, swaps)
    end for
    for end <- n - 1 downto 1 do
        SWAP(A[0], A[end])
        swaps <- swaps + 1
        SIFTDOWN(A, 0, end, swaps)
    end for
    return A
end procedure

procedure SIFTDOWN(A, root, size, swaps)
    while 2*root + 1 < size do
        child <- 2*root + 1
        if child + 1 < size and A[child+1] > A[child] then
            child <- child + 1
        end if
        if A[root] >= A[child] then
            break
        end if
        SWAP(A[root], A[child])
        swaps <- swaps + 1
        root <- child
    end while
end procedure
```

Solution Approach: Heap Sort first builds a max-heap from the array, so the largest value sits at the root. It then repeatedly swaps the root with the last unsorted position and shrinks the heap by one, calling siftDown to restore the heap property each time. This process places one more element into its final sorted position with every pass.



Problem 6: Binary Tree

Testing:

```
Original: 34 7 23 32 5 62 14 19
Heap:  62 32 34 19 5 23 14 7
end = 7 heap: 34 32 23 19 5 7 14
sorted: 62
end = 6 heap: 32 19 23 14 5 7
sorted: 34 62
end = 5 heap: 23 19 7 14 5
sorted: 32 34 62
end = 4 heap: 19 14 7 5
sorted: 23 32 34 62
end = 3 heap: 14 5 7
sorted: 19 23 32 34 62
end = 2 heap: 7 5
sorted: 14 19 23 32 34 62
end = 1 heap: 5
sorted: 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
```

Question: Is Heap Sort stable? Explain your answer in your report using a small example array that contains two equal values.

Heap Sort is not stable. For example, if you have an array like [3a, 3b, 1], where 3a and 3b are the same number but labeled so they can be told apart. Heap Sort only cares about the size of the numbers when it swaps things around, not which one came first. So it's possible for 3b to end up placed before 3a in the final result, even though 3a was originally in front.

Problem 7 [Comparing the Six Algorithms]

Pseudocode:

```
COMPARE-ALL(A, sortedCopy, reversedCopy, comparisons)
    call bubbleSort(A, comparisons)
    print comparisons
    call bubbleSort(sortedCopy, comparisons)
    print comparisons
    call bubbleSort(reversedCopy, comparisons)
    print comparisons

    call insertionSort(A, comparisons)
    print comparisons
    call insertionSort(sortedCopy, comparisons)
    print comparisons
    call insertionSort(reversedCopy, comparisons)
    print comparisons

    call selectionSort(A, comparisons)
    print comparisons
    call selectionSort(sortedCopy, comparisons)
    print comparisons
    call selectionSort(reversedCopy, comparisons)
    print comparisons

    call quickSort(A, comparisons)
    print comparisons
    call quickSort(sortedCopy, comparisons)
    print comparisons
    call quickSort(reversedCopy, comparisons)
    print comparisons

    call mergeSort(A, comparisons)
    print comparisons
    call mergeSort(sortedCopy, comparisons)
    print comparisons
    call mergeSort(reversedCopy, comparisons)
    print comparisons

    call heapSort(A, comparisons)
    print comparisons
    call heapSort(sortedCopy, comparisons)
    print comparisons
    call heapSort(reversedCopy, comparisons)
    print comparisons

for size <- 1000, 5000, 10000 do
    create randomArray of length size, values 1 to 100000
    for each sortFunction in {bubbleSort, insertionSort,
```

```
        selectionSort, quickSort, mergeSort, heapSort} do
        copy randomArray into testArray
        start <- current time
        call sortFunction(testArray)
        end <- current time
        print end - start
        if size = 10000 then
            print first 10 elements of testArray
        end if
    end for
end for
```

Solution Approach: This program reuses the sorting functions from Problems 1 through 6, with all the step-by-step printing taken out, keeping only the comparison and swap counters. Each algorithm runs on three arrays, the original one, a sorted copy, and a reversed copy, to build the comparison table. Then, using chrono, each algorithm is timed on freshly made random arrays of size 1000, 5000, and 10000 to compare how fast they run as the input gets bigger.

Testing:

```
Problem 1: Original comparisons: 25
Problem 1: Sorted comparisons: 7
Problem 1: Reversed comparisons: 28
Problem 2: Original comparisons: 20
Problem 2: Sorted comparisons: 7
Problem 2: Reversed comparisons: 28
Problem 3: Original comparisons: 28
Problem 3: Sorted comparisons: 28
Problem 3: Reversed comparisons: 28
Problem 4: Original comparisons: 14
Problem 4: Sorted comparisons: 28
Problem 4: Reversed comparisons: 28
Problem 5: Original comparisons: 17
Problem 5: Sorted comparisons: 12
Problem 5: Reversed comparisons: 12
Problem 6: Original swaps: 19 comparisons: 30
Problem 6: Sorted swaps: 22 comparisons: 32
Problem 6: Reversed swaps: 16 comparisons: 28
```

```
 Problem 1 Sort Timing
Size 1000: 1444 microseconds
Size 5000: 39466 microseconds
First 10 sorted: 7 10 17 27 34 42 43 48 71 83
Size 10000: 173989 microseconds

 Problem 2 Sort Timing
Size 1000: 497 microseconds
Size 5000: 12404 microseconds
First 10 sorted: 7 10 17 27 34 42 43 48 71 83
Size 10000: 49473 microseconds

 Problem 3 Sort Timing
Size 1000: 954 microseconds
Size 5000: 24093 microseconds
First 10 sorted (Selection): 7 10 17 27 34 42 43 48 71 83
Size 10000: 97715 microseconds

 Problem 4 Sort Timing
Size 1000: 64 microseconds
Size 5000: 370 microseconds
First 10 sorted: 7 10 17 27 34 42 43 48 71 83
Size 10000: 779 microseconds

 Problem 5 Sort Timing
Size 1000: 96 microseconds
Size 5000: 533 microseconds
First 10 sorted: 7 10 17 27 34 42 43 48 71 83
Size 10000: 1122 microseconds

 Problem 6 Sort Timing
Size 1000: 93 microseconds
Size 5000: 548 microseconds
First 10 sorted: 7 10 17 27 34 42 43 48 71 83
Size 10000: 1232 microseconds
```

Question: Which algorithms grow like n^2 and which grow like $n \log n$? Do your measurements agree with the theory? Which algorithm would you choose for a large random array, and which for an array that is already almost sorted? Explain your choices.

Bubble Sort, Insertion Sort, and Selection Sort grow like n^2, since their times got about 65 to 95 times bigger going from size 1000 to size 10000. Quick Sort, Merge Sort, and Heap Sort grow like n log n instead, since their times only went up about 9 to 15 times over that same range. My results match the theory, since making the input 10 times bigger should make an n^2 algorithm run about 100 times slower, but should only make an n log n algorithm run about 13 times slower.

For a large random array, I would use Merge Sort or Heap Sort, since they always run in n log n time no matter how the numbers are arranged. For an array that is already almost sorted, I would use Insertion Sort instead, since it barely has to do any work when things are already in order. Quick Sort would actually be a bad choice here, since an already sorted array is its worst case for it.